TALLER PRÁCTICO FORMATIVO: ARQUITECTURA WEB CLIENTE · SESIONES 26 A 28

Manipulación del DOM, Componentes Reutilizables, Eventos, Promesas y Animaciones en Vanilla JS

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN GENERAL — IDENTIFICACIÓN DEL CURSO

CÓDIGO:	09810 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	12378 / 12377 (Grupos 001 y 003)	**CARÁCTER:**	Práctico Formativo y Aplicado
MODALIDAD:	Equipos de 2 Estudiantes	**DURACIÓN:**	2 Horas en Clase + Cierre Autónomo
POLÍTICA IAG:	Nivel 2: Asistencia de Consulta y Depuración	**PONDERACIÓN:**	Formativo (Alineación RAA 3 / Preparación Parcial 3)
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto: Modo Estricto y Manipulación Reactiva del DOM

En el desarrollo web moderno, las interfaces de usuario no se conciben como documentos estáticos impresos, sino como **aplicaciones reactivas en memoria**. Comprender las bases de Vanilla JavaScript permite al ingeniero construir interfaces de alto rendimiento, dominar el ciclo de vida del navegador y comprender con precisión qué ocurre por debajo de cualquier framework contemporáneo.

Directiva de Seguridad: Modo Estricto (”use strict”)

El Modo Estricto transforma errores silenciosos y conductas inseguras en excepciones explícitas inmediatas (`throw error`). Prohíbe variables globales accidentales (`x = 10` sin declarar lanza `ReferenceError`), asigna `undefined` a `this` en funciones sueltas y previene colisiones en compiladores JIT (V8). En módulos ES6 nativos (`<script type="module">`), el modo estricto está habilitado de forma automática e irrevocable.

1.1 Mutación Limpia vs. Antipatrones Inseguros

El uso indiscriminado de `element.innerHTML = "..."` destruye listeners previos, genera vulnerabilidades de inyección de código (**Cross-Site Scripting - XSS**) y satura el motor gráfico con múltiples ciclos de *Reflow* (cálculo geométrico) y *Repaint* (redibujado de píxeles).

```
"use strict";

// ANTIPATRON: Inseguro contra ataques XSS y costoso en render
function renderItemInseguro(tarea, contenedor) {
  contenedor.innerHTML += `<div class="item">${tarea.titulo}</div>`;
}

// PATRON RECOMENDADO: Nodos en memoria con sanitizacion nativa
function renderItemSeguro(tarea, contenedor) {
```

```
  const item = document.createElement("article");
  item.className = "task-item";
  item.dataset.id = tarea.id;

  const title = document.createElement("span");
  title.textContent = tarea.titulo; // 100% inmune a XSS

  const btn = document.createElement("button");
  btn.className = "btn-delete";
  btn.textContent = "Eliminar";

  item.append(title, btn);
  contenedor.appendChild(item);
}
```

2 Arquitectura de Componentes Reutilizables en Vanilla JS

Un componente funcional en Vanilla JavaScript es una función pura que recibe un objeto de propiedades de entrada (**props**) y retorna un nodo del DOM completamente configurado.

- **Paso de Props Descendente (*Props Down*):** Los componentes padres transmiten datos hacia los componentes hijos mediante argumentos.
- **Notificación Ascendente (*Callbacks Up*):** Los componentes hijos nunca mutan el estado central directamente; notifican al padre invocando funciones callback pasadas en sus props (ej. `onDelete(id)`, `onToggle(id)`).
- **Estado Centralizado en el Componente Raíz (`App`):** Toda la lógica de negocio reside en el componente raíz, garantizando una única fuente de verdad (*Single Source of Truth*).

3 Propagación de Eventos y el Patrón de Delegación

Cuando ocurre una interacción física en el navegador, el evento recorre tres fases formales:

1. **Fase de Captura (*Capturing*):** El evento desciende desde `window` y `document` hasta alcanzar el nodo objetivo.
2. **Fase de Destino (*Target*):** Se alcanza el nodo sobre el que se originó el clic (`event.target`).
3. **Fase de Burbujeo (*Bubbling*):** El evento asciende hacia los ancestros por defecto (`bubbles: true`).

Eficiencia de la Delegación de Eventos

En lugar de asociar 500 listeners a 500 botones individuales en memoria RAM, registramos **un único listener en el contenedor padre** (`<ul>`) y filtramos con `event.target.closest(".btn-delete")`. Esto soporta automáticamente nuevos elementos creados dinámicamente sin fugas de memoria.

4 Asincronía: Promesas, Async/Await y Prioridad de Microtareas

El motor JavaScript ejecuta en un único hilo. Las operaciones asíncronas no bloqueantes se gestionan mediante el **Event Loop**:

- **Microtasks (Alta Prioridad):** Las Promesas resueltas (`.then()`, `async/await`) entran a la cola de microtareas y se ejecutan inmediatamente apenas se vacía el Call Stack.
- **Macrotasks (Baja Prioridad):** Los temporizadores (`setTimeout`) esperan a que la cola de microtareas esté totalmente vacía antes de ejecutarse.

5 Animaciones con Web Animations API y Trazado SVG

Para dinamizar la interfaz sin degradar el rendimiento, utilizamos:

- **Web Animations API (WAAPI):** `elem.animate([...], {duration: 300})` ejecutado sobre `transform` y `opacity` en GPU.
- **Animación Vectorial SVG:** Manipulación de `stroke-dashoffset` y atributos geométricos interactivos.

6 Reto Práctico: Dashboard DevPulse y Rúbrica Oficial

Cada equipo construirá la aplicación modular de observabilidad **DevPulse** en Vanilla JavaScript puro (ES6+) partiendo de un contenedor raíz limpio (`<div id='root'></div>`).

Requerimientos Principales del Proyecto:

- **Renderizado Dinámico y Seguro:** Renderizar catálogo de microservicios (`Auth`, `Payments`, `DB`) mediante `createElement()` y `textContent`, 100 % inmune a XSS. Prohibido `innerHTML`.
- **Arquitectura de Componentes Funcionales:** Desacoplamiento en componentes puros (`ServiceCard`, `MetricGaugeSVG`, `FilterBar`) con *Props Down* y *Callbacks Up*.
- **Delegación de Eventos:** Un único listener delegado sobre el contenedor `#services-grid` gestionando acciones de reinicio, cortacircuitos y eliminación con `closest('[data-action]')`.
- **Asincronía y Simulación de Fallos:** Servicio de sondeo con Promesas y `async/await`, controlando estados *Pending* (carga visual), *Fulfilled* y *Rejected* (captura de errores 500/503 con `try...catch...finally`).
- **Métricas SVG y Web Animations API:** Gráficos radiales vectoriales SVG con actualización de `stroke-dashoffset` y animaciones a 60 FPS con `element.animate()` en GPU.

Rúbrica de Evaluación Modularizada (Escala 0.0 a 5.0):

Criterio	Peso	Indicador de Logro
Manipulación del DOM y Erradicación XSS	20 %	Creación 100 % programática con `createElement()`, inserción segura con `textContent` y reemplazo atómico con `replaceChildren()`. Cero XSS.
Arquitectura de Componentes y Estado	20 %	Componentes funcionales puros desacoplados, flujo Props Down / Callbacks Up y estado centralizado único sin variables globales.
Delegación de Eventos y Memoria	20 %	Un único listener padre delegado con filtrado mediante `closest()` y `matches()`, soportando mutaciones dinámicas sin fugas.
Asincronía con Promesas y Async/Await	20 %	Control riguroso de estados (Pending, Fulfilled, Rejected), manejo de excepciones con `try...catch...finally` y simulación de latencia de red.
Animaciones WAAPI, SVG y Modo Estricto	20 %	Transiciones fluidas a 60 FPS con Web Animations API, medidor radial interactivo SVG y cumplimiento estricto de `"use strict";`.

Portal Interactivo de Aprendizaje Web

Este taller cuenta con un simulador interactivo completo en el portal Docusaurus del curso (`portal/docs/talleres/taller-dom-vanillajs-componentes`), con un Sandbox de código en vivo y simuladores visuales en tiempo real del Event Loop, de la propagación de eventos y del ciclo de vida de componentes.